CarbonIQ ML Pipeline

Deep Dive Notebook Explanation

CarbonIQ_ML_Clean.ipynb

College AIML Mini-Project

Reproducible Machine Learning Pipeline Documentation

Generated: October 2026

Based on: CarbonIQ_ML_Clean.ipynb (29 cells, 13 sections)

Table of Contents

1. Notebook Overview & Structure 3
2. Section A: Project Overview & Objective 4
3. Section B: Imports & Configuration 4
4. Section C: Dataset Loading & Validation 5
5. Section D: Monthly Column Identification 6
6. Section E: ML Window Construction 7
7. Section F: Minimal EDA & Feature Preparation 8
8. Section G: Chronological Train/Test Split 9
9. Section H: Model Comparison (5 Models) 10
10. Section I: Final Model Selection & Evaluation 11
11. Section J: Deterministic Carbon Calculator 12
12. Section K: Integrated Forecasting + Carbon Pipeline 13
13. Section L: Pipeline Smoke Tests 14
14. Section M: Model Metadata, Serialization & Backend Export 15
15. Key Design Decisions & Rationale 16
16. Reproducibility Guarantees 17
17. Change Report Summary 18

1. Notebook Overview & Structure

1.1 What Is This Notebook?

CarbonIQ_ML_Clean.ipynb is the cleaned, reproducible machine learning pipeline for the CarbonIQ project. It transforms raw Pune household survey data into a trained Gradient Boosting model for electricity consumption forecasting, with a deterministic carbon calculator layered on top. The notebook executes top-to-bottom from a fresh runtime with no manual variable restoration needed.

1.2 Notebook Statistics

Property	Value
Total Cells	29 (13 markdown + 16 code)
Sections	13 (labeled A through M)
Original Cells	~81 (reduced by 64%)
Execution Time	~30-60 seconds on modern hardware
Data Size	1,872 households x 26 monthly columns
ML Samples	5,347 train / 1,434 test
Output Artifacts	3 files (model.joblib, metadata.json, backend_package.zip)

1.3 Section Map (A-M)

Sec	Title	Cells	Purpose
A	Project Overview and Objective	1 markdo	Problem framing, scope, key decisions
B	Imports and Configuration	1 code	Dependencies, paths, emission factor, reproducibility
C	Dataset Loading and Essential Validat	1 code	Load CSV, verify shape, identify monthly columns
D	Monthly Column Identification and Chr	1 code	Sort 26 monthly columns chronologically
E	Construction of Genuine Consecutive-	1 code	Build 3-month windows (t-2, t-1, target)
F	Minimal EDA and Feature Preparation	1 code	Target statistics, train/test split preparation
G	Chronological Train/Test Split	1 code	Split by date: pre-Nov 2019 train, Nov 2019-Feb 2020 test
H	Consistent Model Comparison (5 mod	2 code	Naive, Linear, RF, GB, Seasonal GB - same split
I	Final Model Selection and Evaluation	1 code	Pick best MAE, compute final metrics
J	Deterministic Electricity Carbon Calcul	1 code	CEA factor 0.675 kg CO2/kWh, versioned
K	Integrated Forecasting + Carbon Pipel	1 code	End-to-end prediction + emissions function
L	Pipeline Smoke Tests	1 code	4/4 tests: health, predict, carbon, invalid input
M	Model Metadata, Serialization, Backen	4 code	Save model, metadata, create backend package

2. Section A: Project Overview & Objective

2.1 Problem Definition

Build a reproducible ML pipeline that forecasts next-month household electricity consumption from two prior months of genuine consecutive observations, then estimates associated CO2 emissions using a versioned, documented emission factor.

2.2 Scope

- Dataset: Pune Household Survey (1,872 households, 26 monthly electricity columns spanning Jan 2018 - Feb 2020)
- ML Task: Next-month electricity prediction (kWh) using t-2 and t-1 consumption + calendar month
- Train/Test Split: Chronological - train on data before Nov 2019, test on Nov 2019 - Feb 2020
- Models Compared: Naive Persistence, Linear Regression, Random Forest, Gradient Boosting, Seasonal Gradient Boosting
- Carbon Calculation: Deterministic, using CEA v22.0 emission factor (0.675 kg CO2/kWh, FY 2025-26)
- Deliverables: Trained model artifact, metadata, FastAPI backend package with smoke tests

2.3 Key Design Decisions

- Only genuine consecutive 3-month windows are used (no imputation across gaps)
- Extreme observations (e.g., 6,634 kWh) are preserved - they are real survey records
- The Seasonal Gradient Boosting model achieves the lowest MAE (32.12 kWh); standard Gradient Boosting has slightly better RMSE (185.60) and R2 (0.176)
- Carbon calculation is separate from the ML model - the emission factor is configurable and versioned
- Forecasting accuracy is modest (R2 approx 0.17); this is documented, not overstated

3. Section B: Imports & Configuration

3.1 Core Dependencies

The notebook uses only essential, well-maintained libraries:

- pandas, numpy - data manipulation
- scikit-learn - ML models, metrics, pipelines, imputation
- matplotlib, seaborn - visualization (minimal, only where needed)
- joblib - model serialization
- json, pathlib - configuration and paths

3.2 Reproducibility Configuration

```
# Reproducibility
RANDOM_STATE = 42

# Paths
DATA_PATH = "/content/Pune_HHLD_Survey_Data_v2-0-0.csv"
ARTIFACT_DIR = Path("/content/carboniq_artifacts")
ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_PATH = ARTIFACT_DIR / "carboniq_seasonal_gb.joblib"
METADATA_PATH = ARTIFACT_DIR / "carboniq_model_metadata.json"
BACKEND_ZIP_PATH = Path("/content/CarbonIQ_Backend_Package.zip")

# Versioned emission factor (source: Central Electricity Authority, database v22.0)
ELECTRICITY_FACTOR = {
    "kg_co2_per_kwh": 0.675,
    "source": "Central Electricity Authority (CEA)",
    "database_version": "22.0",
    "reporting_period": "FY 2025-26",
    "gas": "CO2"
}
```

3.3 Why These Choices?

Fixed RANDOM_STATE=42 ensures identical results across runs. All paths are configurable at the top. The emission factor is defined as a dictionary with full provenance (source, version, reporting period, gas) so it can be versioned and audited separately from the ML model.

4. Section C: Dataset Loading & Validation

4.1 Data Loading

```
df = pd.read_csv(
    DATA_PATH,
    encoding="cp1252",
    low_memory=False
)
```

```
print(f"Dataset shape: {df.shape}")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
```

4.2 Essential Validation

The notebook performs only essential validation - checking column dtypes and identifying monthly electricity columns. No exhaustive EDA is included (removed during cleanup). The goal is to validate what informs downstream decisions.

4.3 Monthly Column Detection

```
monthly_cols = [
    col for col in df.columns
    if any(month in col for month in [
        "Jan_", "Feb_", "Mar_", "April_", "May_", "June_",
        "July_", "Aug_", "Sep_", "Oct_", "Nov_", "Dec_"
    ])
]
print(f"Monthly electricity columns found: {len(monthly_cols)}")
```

5. Section D: Monthly Column Identification & Ordering

5.1 Chronological Sorting Logic

Monthly columns in the raw CSV are not guaranteed to be in chronological order. The notebook defines a month_order mapping and sorts columns by (year, month):

```
month_order = {
    "Jan": 1, "Feb": 2, "Mar": 3, "April": 4,
    "May": 5, "June": 6, "July": 7, "Aug": 8,
    "Sep": 9, "Oct": 10, "Nov": 11, "Dec": 12
}

def month_key(col):
    parts = col.split("_")
    return (int(parts[1]), month_order[parts[0]])

monthly_cols_chrono = sorted(monthly_cols, key=month_key)
```

5.2 Verification

After sorting, the notebook prints the chronological column order and non-null counts per month to verify data coverage across the Jan 2018 - Feb 2020 period.

6. Section E: ML Window Construction (Core Logic)

6.1 Genuine Consecutive 3-Month Windows

This is the heart of the data preparation. For each household and each valid 3-month window where ALL THREE months have non-null values, a training record is created:

```
records = []

for household_idx, row in df[monthly_cols_chrono].iterrows():
    for i in range(2, len(monthly_cols_chrono)):
        prev_2 = row[monthly_cols_chrono[i - 2]]
        prev_1 = row[monthly_cols_chrono[i - 1]]
        target = row[monthly_cols_chrono[i]]

        if pd.notna(prev_2) and pd.notna(prev_1) and pd.notna(target):
            records.append({
                "household_id": household_idx,
                "month_t_minus_2": monthly_cols_chrono[i - 2],
                "month_t_minus_1": monthly_cols_chrono[i - 1],
                "target_month": monthly_cols_chrono[i],
                "electricity_t_minus_2": prev_2,
                "electricity_t_minus_1": prev_1,
                "electricity_target": target
            })

ml_df = pd.DataFrame(records)
```

6.2 Why This Approach?

- No imputation - only genuine observations where all 3 months exist
- Preserves extreme values (6,634 kWh spike) as real data
- Creates target_month_num feature (1-12) from target column name for seasonality
- Results in 6,781 total samples before train/test split

6.3 Target Statistics

The target variable (electricity_target) statistics:
- Count: 6,781
- Mean: ~110 kWh
- Std: ~180 kWh
- Min: 0 kWh
- Max: 6,634 kWh (preserved outlier)
- 25th/50th/75th percentiles: ~35/70/130 kWh

7. Section F: Minimal EDA & Feature Preparation

7.1 Feature Engineering

The notebook keeps feature engineering minimal and transparent. Features are:
1. electricity_t_minus_2: Consumption 2 months ago (kWh)
2. electricity_t_minus_1: Consumption 1 month ago (kWh)
3. target_month_num: Calendar month of target (1-12)

No rolling statistics, no polynomial features, no external data. The Seasonal Gradient Boosting model captures seasonality through the target_month_num feature.

7.2 Train/Test Split Preparation

The chronological split is implemented by extracting the year-month from the target_month column and splitting at November 2019 (period 201911). This ensures no temporal leakage.

8. Section G: Chronological Train/Test Split

8.1 Split Strategy

The split is purely chronological - critical for time-series data to prevent data leakage:

```
# Split by target month period
train_data = ml_df[ml_df["target_month_period"] < 201911].copy()
test_data = ml_df[ml_df["target_month_period"] >= 201911].copy()

print(f"Train samples: {len(train_data)}")
print(f"Test samples: {len(test_data)}")
print(f"Train period: up to Oct 2019")
print(f"Test period: Nov 2019 - Feb 2020")
```

8.2 Split Statistics

Property	Value
Train samples	5,347
Test samples	1,434
Train period	Jan 2018 - Oct 2019
Test period	Nov 2019 - Feb 2020
Split type	Chronological (no random shuffle)

9. Section H: Model Comparison (5 Models)

9.1 Models Evaluated

Model	Description	Configuration
Naive Persistence	Predict t-1 as target (baseline)	No training
Linear Regression	Linear model with imputation	Pipeline: Imputer + LinearRegression
Random Forest	Ensemble of decision trees	RandomForestRegressor, n_estimators=200
Gradient Boosting	Standard GBRegressor	GradientBoostingRegressor, n_estimators=200
Seasonal Gradient Boostir	GB with month_num feature	Same as GB but includes target_month_num

9.2 Evaluation Protocol

All models use the EXACT SAME train/test split (chronological). Pipeline includes SimpleImputer(strategy="median") for robustness. Metrics computed: MAE, RMSE, R2 on test set only.

9.3 Results Summary

Model	MAE (kWh)	RMSE (kWh)	R2
Naive Persistence	38.45	212.31	0.089
Linear Regression	35.12	198.45	0.123
Random Forest	33.87	192.10	0.145
Gradient Boosting	32.45	185.60	0.176
Seasonal Gradient Boosting	32.12	186.39	0.169

9.4 Key Insight

Seasonal Gradient Boosting achieves the LOWEST MAE (32.12 kWh) which is the primary metric for this use case (absolute error matters more than squared error for household forecasting). Standard GB has slightly better RMSE and R2 but higher MAE. The seasonal feature (target_month_num) helps capture the annual consumption cycle.

10. Section I: Final Model Selection & Evaluation

10.1 Selected Model

Seasonal Gradient Boosting is selected based on lowest MAE. The notebook re-trains on the full training set and evaluates on the held-out test set to produce final metrics:

10.2 Final Metrics

Metric	Value

MAE	32.1234 kWh
RMSE	186.3864 kWh
R2	0.1689

10.3 Interpretation

R2 = 0.1689 means the model explains ~17% of variance in next-month electricity consumption. MAE = 32.12 kWh means average absolute error is ~32 kWh (on typical ~110 kWh consumption). This is modest but honest performance - the model captures seasonal trends but household-specific variation dominates. The limitations are explicitly documented in metadata.

11. Section J: Deterministic Carbon Calculator

11.1 Design Principle

Carbon calculation is DELIBERATELY separate from the ML model. The emission factor is a policy/regulatory value, not something to be learned. This separation allows:
- Factor updates without model retraining
- Clear provenance and versioning
- Transparent, verifiable calculations
- Different factors for different regions/periods

11.2 Implementation

```
def calculate_co2(electricity_kwh, factor_kg_per_kwh=0.675):
    """Deterministic CO2 calculation from electricity consumption."""
    electricity_kwh = max(0.0, electricity_kwh)  # Non-negative
    return round(electricity_kwh * factor_kg_per_kwh, 3)

# Example
predicted_kwh = 117.11
predicted_co2 = calculate_co2(predicted_kwh)
# Result: 79.049 kg CO2
```

11.3 Emission Factor Provenance

Property	Value
Value	0.675 kg CO2/kWh
Source	Central Electricity Authority (CEA), Government of India
Database Version	22.0
Reporting Period	FY 2025-26
Gas	CO2 (not CO2e)
Scope	Grid electricity only (Scope 2)

12. Section K: Integrated Pipeline

12.1 End-to-End Function

The notebook combines the trained model with the carbon calculator into a single predict_and_calculate function that mimics the backend API behavior:

```
def predict_and_calculate(electricity_t_minus_2, electricity_t_minus_1, target_month_num):
    """Full pipeline: predict electricity -> calculate CO2."""
    # Prepare input in correct feature order
    X = pd.DataFrame([{
        "electricity_t_minus_2": electricity_t_minus_2,
        "electricity_t_minus_1": electricity_t_minus_1,
        "target_month_num": target_month_num
```

```
    }])

    # Predict
    predicted_kwh = max(0.0, float(model.predict(X)[0]))
    predicted_kwh = round(predicted_kwh, 2)

    # Calculate CO2
    predicted_co2 = round(predicted_kwh * ELECTRICITY_FACTOR["kg_co2_per_kwh"], 3)

    return {
        "predicted_electricity_kwh": predicted_kwh,
        "predicted_co2_kg": predicted_co2,
        "emission_factor": ELECTRICITY_FACTOR
    }

# Test
result = predict_and_calculate(92, 114, 3)
# {"predicted_electricity_kwh": 117.11, "predicted_co2_kg": 79.049, ...}
```

13. Section L: Pipeline Smoke Tests

13.1 Test Coverage

Four automated smoke tests verify the complete pipeline works end-to-end:

Test	Purpose
Health Check	Model loads correctly, metadata accessible
Prediction	Valid inputs produce reasonable kWh + CO2 output
Carbon Calculation	Deterministic CO2 matches manual calculation
Invalid Input Rejection	Negative kWh, month > 12, missing fields rejected

13.2 Verification Results

All 4/4 smoke tests pass. The notebook also verifies:
- Model reload produces identical predictions (atol < 1e-9)
- Metadata evaluation matches computed metrics exactly
- Backend package contains all 4 required files
- API health, prediction, carbon calc, invalid input rejection all pass

This ensures the artifacts are consistent and deployable.

14. Section M: Model Metadata, Serialization & Backend Export

14.1 Model Serialization

```
# Save model
joblib.dump(final_model, MODEL_PATH)

# Verify reload
reloaded_model = joblib.load(MODEL_PATH)
reload_pred = reloaded_model.predict(X_test)
original_pred = final_model.predict(X_test)
assert np.allclose(reload_pred, original_pred, atol=1e-9)
```

14.2 Metadata JSON Structure

The metadata.json contains everything needed to run the model and understand its provenance:

- project, model_name, model_type, model_file
- features (ordered list), target, target_unit
- training: samples, test_samples, cutoff, periods
- evaluation: MAE, RMSE, R2
- emission_factor: kg_co2_per_kwh, source, database_version, reporting_period, gas
- limitations (array of strings)

- software: python_version, scikit_learn_version
- model_parameters (full sklearn get_params())

14.3 Backend Package Creation

The final step creates CarbonIQ_Backend_Package.zip containing:
1. app.py - FastAPI application
2. carboniq_seasonal_gb.joblib - trained model
3. carboniq_model_metadata.json - metadata
4. requirements.txt - pinned dependencies

This zip can be deployed directly to any Python 3.10+ environment.

15. Key Design Decisions & Rationale

15.1 Data Decisions

- **No imputation across gaps:** Only genuine consecutive 3-month windows used
- **Extreme values preserved:** 6,634 kWh spike kept as real survey data
- **Chronological split:** Prevents temporal leakage, mimics real deployment
- **Minimal features:** Only t-2, t-1, month_num - interpretable, low overfit risk

15.2 Model Decisions

- **Seasonal GB over standard GB:** Lower MAE (32.12 vs 32.45) - better for household forecasting
- **Fixed hyperparameters:** No tuning - reproducible, avoids overfitting to test set
- **joblib serialization:** Standard sklearn format, fast load, includes preprocessing
- **Separate carbon calculation:** Policy value, not learned; versioned independently

15.3 Documentation Decisions

- **Honest limitations:** R2=0.17 documented prominently, not hidden
- **Versioned emission factor:** CEA v22.0, FY 2025-26 in metadata and API response
- **Reproducible notebook:** Runs top-to-bottom from fresh runtime
- **Clean notebook:** 29 cells vs original 81 - removed redundancy, debugging, duplicates

16. Reproducibility Guarantees

16.1 What "Reproducible" Means Here

- Same random seed (42) -> identical train/test split, model initialization
- Same data path -> identical dataset (CSV not modified)
- Same code execution order -> identical artifacts
- No external API calls, no manual steps, no session-dependent variables
- All objects (df, ml_df, train_data, test_data, final_model) created in sequence
- Model reload verification: predictions match to 1e-9 tolerance

16.2 Requirements for Reproduction

- Python 3.10+ (notebook uses 3.13.16)
- scikit-learn 1.6.1 (pinned in requirements.txt)
- pandas 2.2.3, numpy 2.1.3, joblib 1.6.0
- Pune_HHLD_Survey_Data_v2-0-0.csv at DATA_PATH
- ~500MB RAM, ~30 seconds compute time

16.3 Verification Checklist

- Train samples = 5,347
- Test samples = 1,434
- MAE = 32.1234 (to 4 decimals)
- RMSE = 186.3864 (to 4 decimals)
- R2 = 0.1689 (to 4 decimals)
- Model file size ~200 KB
- Metadata JSON contains all fields
- Backend zip contains 4 files

17. Change Report Summary (From CHANGE_REPORT.md)

17.1 Cell Reduction

Original notebook: ~81 cells (execution_count up to 81)
Cleaned notebook: 29 cells
Reduction: ~64% fewer cells
Structure: Organized into 13 clear sections (A-M) as specified

17.2 Categories of Removal

- **Redundant EDA & Investigation:** Duplicate missing-value checks, column listings, outlier inspections, bill/provenance investigation
- **Runtime Recovery/Debugging:** Cells created to restore lost variables after session resets
- **Duplicate Model Evaluation:** Separate Linear/RF/GB/Seasonal GB cells merged into unified comparison
- **Duplicate Carbon Calculator:** First and corrected versions consolidated
- **Duplicate Backend Packaging:** Initial and final saves merged into single authoritative artifacts

17.3 What Was Preserved (Technical Correctness)

- Original dataset untouched - no modifications to source CSV
- Extreme observations preserved - 6,634 kWh spike kept (documented, not removed)
- Genuine consecutive-month windows only - 3-month windows with all values present
- Chronological test period - Nov 2019 to Feb 2020 (periods 201911-202002)
- Same train/test split for all models - 5,347 train / 1,434 test
- Feature definitions preserved - electricity_t_minus_2, electricity_t_minus_1, target_month_num
- Seasonal GB has lowest MAE (32.12) - standard GB has better RMSE (185.60) and R2 (0.176)
- Metrics computed from actual predictions - not hard-coded
- Carbon calculation separate from ML model - emission factor is configurable
- Emission factor provenance preserved - CEA v22.0, FY 2025-26, 0.675 kg CO2/kWh
- Reporting period limitation documented - factor period differs from ML data period
- No overstated accuracy claims - R2 approx 0.17 explicitly noted as modest

17.4 Scope Not Expanded

- No frontend built
- No new features added
- No hyperparameter tuning beyond original config
- No invented data, units, or assumptions
- No outlier removal or data modification